

What is JSX?

→ JSX stands for Javascript XML. It's syntax extension for javascript that look like HTML but is actually Javascript code.

→ React convert JSX into regular Javascript functions.

// This JSX code

```
const element = <h1>Hello</h1>;
```

// Get converted to Javascript

```
const element = React.createElement  
('h1', null, 'Hello');
```

Benefits of JSX

- Readable
- Safe
- Powerful
- Fast

HTML

class = "myClass"

for = "input"

onclick = "doSome()"

style = "color: red;"

self-closing:

<div class="container"

onclick="handleClick()"

<input for="name"

style = "margin: 10px;"

</div>

JSX

className = "my"

htmlFor = "input"

onclick = {doSome}

style = {{color: 'red'}}

self-closing:

<div className="container"

onClick={handleClick}

<input htmlFor="name"

style = {{margin: '10px'}}

</div>

Embedding Expression in JSX

```
const name = "John";
```

```
const element =
```

```
<div>
```

```
<h1> Hello, {name} </h1>
```

```
</div>
```

```
};
```


You can put any valid Javascript expression inside curly braces `{ }`

JSX Must One Parent Element

```
function Component() {
```

```
  return (
```

```
    <h1>Title</h1>
```

```
    <p>Description</p>
```

```
  );
```

```
}
```

// wrong

```
function Component() {
```

```
  return
```

```
    <div>
```

```
      <h1>Title</h1>
```

```
      <p>Description</p>
```

```
    </div>
```

```
  );
```

// correct

Function Component

A functional Component is a JavaScript function that return JSX. Its modern way to create components in React

Method 1:

```
function Welcome() {  
  return <h1>Hello</h1>  
}
```

Method 2:

```
const Welcome = () =>  
  <h1>Hello</h1>;
```


Using Props in Functional Component:

props (properties) → are like function parameters

```
function Greeting(props) {  
  return <h1>Hello, {props.name}</h1>  
}
```

// with destructuring

```
function Greeting({name}) {  
  return <h1>Hello, {name}</h1>;  
}
```

// Usage

```
<Greeting name = "Alice"/>
```